AI-ASSISTANT-CODING-LAB-23.4

Name: G. Kruthik Roshan
Batch: 41
Roll No: 2303A52339

End-to-End Application Development

Task 1 – Problem Identification & AI-Assisted Ideation

Task Description: Identify and refine a real-world problem using AI for a city council that wants to digitize a manual process. Finalize a problem statement, target users, expected impact, and a list of key features.

Prompt Used:

Act as a product manager for a smart city initiative. The city council wants to digitize a manual process. Suggest 3 problem statements. Select one (a Civic Complaint Management System), refine it, and provide a list of key features, target users, and expected impact.

Code:

Lab23_tests.py U ×

Lab23_tests.py > ...

```python
"""Unit tests for Lab 23 capstone complaint management system."""

import os
import tempfile
import unittest

from Lab23_capstone.service import ComplaintService
from Lab23_capstone.storage import JSONStorage


class TestComplaintService(unittest.TestCase):
    """Validate complaint service behavior and edge cases."""

    def setUp(self) -> None:
        self.temp_dir = tempfile.TemporaryDirectory()
        self.data_file = os.path.join(self.temp_dir.name, "lab23_test_data.json")
        self.service = ComplaintService(JSONStorage(self.data_file))

    def tearDown(self) -> None:
        self.temp_dir.cleanup()

    def test_create_complaint_success(self) -> None:
        complaint = self.service.create_complaint(
            citizen_name="Ananya Rao",
            area="Ward 5",
            issue_type="Garbage Overflow",
            description="Bin has not been cleared for 4 days.",
            priority="High",
```

```python
class TestComplaintService(unittest.TestCase):
    def test_persistence_across_service_reloads(self) -> None:

        self.assertEqual(len(complaints), 1)
        self.assertEqual(complaints[0]["citizen_name"], "Rekha")

    def test_dashboard_metrics_counts(self) -> None:
        self.service.create_complaint(
            citizen_name="Ravi",
            area="Ward 1",
            issue_type="Garbage",
            description="Garbage pickup delayed.",
            priority="Critical",
        )
        self.service.create_complaint(
            citizen_name="Pooja",
            area="Ward 1",
            issue_type="Drainage",
            description="Drain blockage on main road.",
            priority="High",
        )

        self.service.update_status(1, "In Progress")
        self.service.update_status(2, "Resolved")

        metrics = self.service.dashboard_metrics()
```

Output :

```
Smart City Complaint and Waste Tracking Platform
1. Register new complaint
2. View all complaints
3. Filter complaints
4. Update complaint status
5. Assign staff
6. Dashboard metrics
7. Exit
Select an option: 1
Citizen Name: virat
Area/Ward: warangal
Issue Type (e.g., Garbage, Pothole): garbage
Description: not good
Select priority:
1. Low
2. Medium
3. High
4. Critical
Select option: 3
Complaint created successfully.
[#1] garbage | Area: warangal | Priority: High | Status: Open
Citizen: virat | Assigned: Unassigned | Created: 2026-04-12 13:37:10 | Updated: 2026-04-12 13:37:10
Description: not good
--------------------------------------------------------------------------------
```

Justification:

I chose the Civic Issue Reporting system because it addresses a very common communication gap between residents and municipal authorities. It's a highly relatable problem, and digitizing it offers immediate, measurable benefits (like tracking visibility), which makes it a perfect candidate for an end-to-end software development project.

Task 2 – System Planning & Architecture Design

Task Description: Design the system architecture with AI support, including frontend, backend, database components, data flow, and database schema for the selected smart service platform.

Prompt Used:

Act as a system architect. Design the architecture for the "CivicFix" complaint system. Suggest frontend, backend, and database components. Define the data flow. Propose a basic database schema for users and complaints.

Architecture & Schema (Text-based):

- **Frontend:** CLI (Command Line Interface) for the prototype (expandable to React.js web app later).
- **Backend:** Python application using modular functions for core logic (Input handling, Ticket management).
- **Database:** In-memory Python lists/dictionaries (simulating a NoSQL JSON database or SQLite for production).
- **Data Flow:** User inputs data via CLI -> Backend validates input -> Backend assigns a unique Ticket ID -> Data stored in the dictionary -> User receives confirmation.
- **Database Schema:**
  - Users: {user_id: INT, name: STR, role: STR [citizen, admin]}
  - Complaints: {ticket_id: INT, citizen_name: STR, category: STR, description: STR, status: STR [Open, In-Progress, Resolved]}

Justification:

I opted for a modular text-based CLI and in-memory dictionary storage for this prototype to ensure rapid development and testing within the lab timeframe. The architecture is deliberately decoupled, meaning if I want to connect a real SQLite database or build a Web UI later, the core Python logic modules won't need to change.

Task 3 – Modular Code Development with AI

Task Description: Develop the core modules of the application using AI. The application must handle user interaction and data processing, including input handling, core logic, and data storage.

Prompt Used:

Write a modular Python program for the CivicFix system. Include an input handling module to register complaints, core logic to view and assign a status, and data storage using a list of dictionaries. Ensure basic error handling and clean structure using functions.

Code:

```
# CivicFix: Complaint Management System
complaints_db = []
ticket_counter = 1

def register_complaint(name, category, description):
    """Core logic to register a new complaint."""
    global ticket_counter
    if not name or not category or not description:
        return "Error: All fields are required."

    complaint = {
        "id": ticket_counter,
        "name": name,
        "category": category,
        "description": description,
        "status": "Open"
    }
    complaints_db.append(complaint)
    ticket_counter += 1
    return f"Success! Complaint registered with Ticket ID: {complaint['id']}"

def view_complaints():
    """Core logic to view all complaints."""
    if not complaints_db:
        return "No complaints found."

    output = "--- All Complaints ---\n"
    for c in complaints_db:
```

```
        output += f"ID: {c['id']} | By: {c['name']} | Cat: {c['category']} | Status: {c['status']}\n"
    return output

def update_status(ticket_id, new_status):
    """Core logic to update complaint status."""
    valid_statuses = ["Open", "In-Progress", "Resolved"]
    if new_status not in valid_statuses:
        return "Error: Invalid status."

    for c in complaints_db:
        if c['id'] == ticket_id:
            c['status'] = new_status
            return f"Ticket {ticket_id} updated to {new_status}."
    return "Error: Ticket ID not found."

# Input handling simulation
if __name__ == "__main__":
    print(register_complaint("John Doe", "Road", "Large pothole on Main St."))
    print(register_complaint("Jane Smith", "Waste", "Garbage not collected."))
    print(view_complaints())
    print(update_status(1, "In-Progress"))
```

Output:

```
Success! Complaint registered with Ticket ID: 1
Success! Complaint registered with Ticket ID: 2
--- All Complaints ---
ID: 1 | By: John Doe | Cat: Road | Status: Open
ID: 2 | By: Jane Smith | Cat: Waste | Status: Open

Ticket 1 updated to In-Progress.
```

Justification:

I structured the code using clearly defined, single-responsibility functions (register_complaint, view_complaints, update_status). This modular approach ensures that input validation and business logic are separated from the data layer (complaints_db). It makes the code much easier to read, test, and debug.

Task 4 – AI-Assisted Testing, Debugging & Optimization

Task Description: Validate and improve the application by generating test cases, identifying possible bugs, suggesting optimizations, and fixing issues to improve performance.

Prompt Used:

Review the Python code for the CivicFix system. Generate 3 test cases. Identify potential bugs (e.g., handling invalid ticket IDs during status updates). Provide an optimized version of the code that uses try-except blocks where necessary and improves input validation.

Test Cases & Bug Identification:

1. **Test Case 1 (Valid Input):** Register complaint with valid strings. *Result: Pass.*
2. **Test Case 2 (Invalid Ticket ID):** Try to update status of Ticket ID 99 or passing a string instead of an int. *Result: Bug identified - code fails silently or crashes if a string is passed instead of an integer ID.*
3. **Test Case 3 (Empty Input):** Pass empty strings to register_complaint. *Result: Pass (handled by existing basic validation).*

Optimized Code:

```
complaints_db = []
ticket_counter = 1

def register_complaint(name: str, category: str, description: str) -> str:
    global ticket_counter
    try:
        # Optimization: Stripping whitespace
        name, category, description = name.strip(), category.strip(), description.strip()
        if not name or not category or not description:
            raise ValueError("Fields cannot be empty or just spaces.")

        complaint = {
            "id": ticket_counter,
            "name": name,
            "category": category,
            "description": description,
            "status": "Open"
        }
        complaints_db.append(complaint)
```

```
        ticket_counter += 1
        return f"Success: Ticket ID {complaint['id']} created."
    except Exception as e:
        return f"Error registering complaint: {e}"

def update_status(ticket_id: int, new_status: str) -> str:
    valid_statuses = ["Open", "In-Progress", "Resolved"]
    try:
        ticket_id = int(ticket_id) # Type enforcement optimization
        if new_status not in valid_statuses:
            raise ValueError("Invalid status provided.")

        # Optimization: using next() with a generator expression for faster lookup O(N)
        complaint = next((c for c in complaints_db if c["id"] == ticket_id), None)
        if complaint:
            complaint["status"] = new_status
            return f"Success: Ticket {ticket_id} updated."
        else:
            return "Error: Ticket ID not found."
    except ValueError as ve:
        return f"Input Error: {ve}"
    except Exception as e:
        return f"System Error: {e}"
```

Output:

(Testing Optimized Code)
Output for update_status("abc", "Resolved"): Input Error: invalid literal for int() with base 10: 'abc'

Justification:

The initial code was fragile regarding data types. If a user entered "one" instead of 1 for an ID, the system would crash. I utilized AI to help implement stricter type hinting, .strip() for input sanitation, and comprehensive try-except blocks. Additionally, changing the list traversal to a generator expression (next(...)) is a great Pythonic optimization that stops searching as soon as the ID is found.

Task 5 – Documentation, Version Control & Reflection

Task Description: Prepare the project for submission by generating a README, adding docstrings/comments, simulating a Git workflow with commit messages, and writing a reflection report.

Prompt Used:

Generate a concise project README.md (Setup + Usage) for the CivicFix app. Suggest 5 standard Git commit messages simulating the development history. Finally, write a short reflection on how AI helped, limitations faced, and future improvements.

README Snippet:

CivicFix: Smart City Complaint System
A lightweight CLI application for reporting and tracking civic issues.

Setup
1. Ensure Python 3.8+ is installed.
2. Clone the repository: `git clone https://github.com/...`
3. Run the application: `python app.py`

Usage
- Developers can import `register_complaint` and `update_status` from `app.py`.
- Data is stored in-memory per session.

Simulated Git Commit Messages:

1. init: setup initial project structure and README
2. feat: add register_complaint function and core data dictionary
3. feat: implement ticket status update logic
4. fix: add input sanitation and try-except error handling
5. docs: add python docstrings to core logic functions

Reflection Report:

- **How AI Helped:** AI was incredibly useful for overcoming "blank page syndrome." It rapidly generated the boilerplate logic and provided excellent suggestions for edge-case testing (like handling string inputs for integer IDs). It also saved time writing the standard README markdown.
- **Limitations Faced:** While the AI wrote the code quickly, I had to be very specific in my prompts to ensure the code remained modular. Initially, the AI tried to put everything into one massive while True: loop, which violated the clean structure requirement. I had to

intervene to force function-based modularity.

- **Improvements Possible:** In the future, this project should be upgraded from in-memory lists to an actual database (like SQLite or PostgreSQL). Additionally, attaching a web-based frontend using a framework like Flask or FastAPI would make it a truly complete, user-facing end-to-end application.

Justification:

Proper documentation and simulated version control are what separate a lab script from professional software. Using AI to generate the README and commit history taught me standard formatting conventions. The reflection highlights a crucial lesson: AI is a powerful assistant, but the developer must remain the "architect" to ensure the output aligns with software engineering best practices (like modularity).